

Feedback

Workshop 4

Workshop 4 feedback

- The fourth workshop looked at adding MPI to the Lissajous program
- The workshop instructions presented the steps required to add MPI
- I will go through the result of applying these steps highlighting the differences from the original program

Workshop 4

```
/******  
Purpose: Program to generate Lissajous curves with MPI parallelisation  
  
Notes: Notation follows https://mathworld.wolfram.com/LissajousCurve.html  
  
        Compile with: mpicc -o lissajous -std=c99 lissajous.c -lm  
        Run with:      mpirun -np N ./lissajous  
  
******/
```

Updated
comment

```
/******  
                        Include header files  
******/
```

```
#include <stdio.h>  
#include <stdlib.h>  
#include <math.h>  
#include <mpi.h>
```

← Include `mpi.h` header

```
/******  
                        Function propotypes  
******/
```

```
void write_to_file(const char *filename, float *x, float *y, int num_pts);
```

```
/******  
                        Main function  
******/
```

```
int main(int argc, char *argv[]) { ← Handle command line arguments
```

```
    /* Define the constant pi */  
    const float PI=3.14159265358979323846;
```

Workshop 4

```
int main(int argc, char *argv[]){

    /* Define the constant pi */
    const float PI=3.14159265358979323846;

    /* Define parameters of the Lissajous curve as constants*/
    const float A=1.0;
    const float B=1.0;
    const float omega_x=1.0;
    const float omega_y=2.0;
    const float delta_x=PI/2.0;
    const float delta_y=0.0;

    /* Number of (x,y) points to calculate */
    const int N=7560;

    /* Arrays to hold values of x and y */
    float x[N], y[N];

    /* Pointers to buffers holding local sub-set of values*/
    float *x_local, *y_local;

    /* Parameter t used in the parametric equations */
    float t;

    int myRank; // Rank of this process
    int nprocs; // Number of MPI processes
    int pts_per_proc; // Number of points per processor
    int start_index; // Starting index for this processor
    const int root=0; // Root process for gather
```

Each MPI process
will store its local
results in these
buffers.

Extra variables for
MPI related
information

Workshop 4

```
int myRank; // Rank of this process
int nprocs; // Number of MPI processes
int pts_per_proc; // Number of points per processor
int start_index; // Starting index for this processor
const int root=0; // Root process for gather
```

```
MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
```

MPI “housekeeping”
Initialise MPI and get
rank and size information

```
/* Calculate number of points for each process to calculate */
pts_per_proc = N/nprocs;
if (myRank==0){
    printf("nprocs= %d \n", nprocs);
    printf("pts_per_proc= %d\n", pts_per_proc);
}
```

Number of
points for each
process
to calculate

```
/* Each process allocates space to store its results */
x_local = (float*) malloc(pts_per_proc*sizeof(float));
y_local = (float*) malloc(pts_per_proc*sizeof(float));
```

Allocate memory
for local buffers

Workshop 4

```
/* Calculate values for x and y by looping over the parameter t */
start_index = myRank * pts_per_proc;
printf("Rank %d calculating %d to %d\n", myRank, start_index, start_index+pts_per_proc-1);
for (int i=0; i<pts_per_proc; i++){
    /* Convert i and N to floats so we do floating point division */
    t = 2.0*PI * (float) (i+start_index) / (float) N;
    x_local[i] = A * sin( omega_x*t + delta_x );
    y_local[i] = B * sin( omega_y*t + delta_y );
}
```

```
/* Gather x results onto root process */
MPI_Gather(x_local, pts_per_proc, MPI_FLOAT,
          x, pts_per_proc, MPI_FLOAT,
          root, MPI_COMM_WORLD);

/* Gather y results onto root process */
MPI_Gather(y_local, pts_per_proc, MPI_FLOAT,
          y, pts_per_proc, MPI_FLOAT,
          root, MPI_COMM_WORLD);
```

$\text{start_index} = rn$

$$t = 2\pi \frac{i + rn}{N}$$

$\text{pts_per_proc} = n$

Each process sends `x_local` and `y_local`
Root process receives values into `x` and `y`

Workshop 4

```
/* Gather x results onto root process */  
MPI_Gather(x_local, pts_per_proc, MPI_FLOAT,  
          x, pts_per_proc, MPI_FLOAT,  
          root, MPI_COMM_WORLD);
```

```
/* Gather y results onto root process */  
MPI_Gather(y_local, pts_per_proc, MPI_FLOAT,  
          y, pts_per_proc, MPI_FLOAT,  
          root, MPI_COMM_WORLD);
```

Only the root process
holds values of **x** and **y**

```
/* Write the results to a text file called output.dat */  
/* x and y are converted to pointers because they are arrays */  
if (myRank==root){  
    write_to_file("output.dat", x, y, N);  
}
```

```
free(x_local);  
free(y_local);
```

Free the memory allocated
for localbuffers

```
MPI_Finalize();
```

Close down MPI

```
return 0;  
}
```

Workshop 4

```
> module load mpi
> mpicc -o lissajous -std=c99 lissajous.c -lm
>
> mpirun -np 2 ./lissajous
nprocs= 2
pts_per_proc= 3780
Rank 0 calculating 0 to 3779
Rank 1 calculating 3780 to 7559
>
> mpirun -np 4 ./lissajous
Rank 2 calculating 3780 to 5669
nprocs= 4
pts_per_proc= 1890
Rank 0 calculating 0 to 1889
Rank 1 calculating 1890 to 3779
Rank 3 calculating 5670 to 7559
>
> mpirun -np 8 ./lissajous
Rank 7 calculating 6615 to 7559
Rank 4 calculating 3780 to 4724
Rank 6 calculating 5670 to 6614
nprocs= 8
pts_per_proc= 945
Rank 0 calculating 0 to 944
Rank 5 calculating 4725 to 5669
Rank 3 calculating 2835 to 3779
Rank 2 calculating 1890 to 2834
Rank 1 calculating 945 to 1889
```

Whole calculation
calculates 7560 values:
0→7559

0→pts_per_proc-1
pts_per_proc→2*pts_per_proc-1

$$t = 2\pi \frac{i + rn}{N}$$

myRank = r

pts_per_proc = n

Workshop 4 key points

- Adding MPI is much more work than adding OpenMP (less suited to incremental parallelism)
- We needed to allocate buffers for storing the local results (values on each processor) so there is a memory overhead
- Extra steps are required:
 - Divide the workload between processors
 - Gather results